Análise Assintótica de Algoritmos

A análise assintótica estuda como o tempo de execução ou o consumo de recursos de um algoritmo cresce em função do tamanho da entrada n, considerando valores suficientemente grandes de n.

1. Ordem crescente de complexidade

A sequência apresentada está em ordem crescente de crescimento assintótico:

Θ(1)

1. **Constante**: Não depende do tamanho da entrada.

Θ(lg n)

2. Logarítmica: Cresce lentamente; comum em buscas por divisão e conquista.

Θ(n)

3. Linear: Cresce proporcionalmente ao tamanho da entrada.

Θ(n lg n)

4. **Linearítmica**: Comum em algoritmos eficientes de ordenação.

Θ(n^2)

5. **Quadrática**: Comum em dois laços aninhados.

Θ(n^2 lg n)

6. **Quadrática-logarítmica**: Combina crescimento quadrático e logarítmico.

Θ(n^3)

7. **Cúbica**: Comum em três laços aninhados.

Θ(2^n)

8. **Exponencial**: O crescimento pode se tornar muito rápido.

Nota: lg n representa o logaritmo de n na base 2, convenção frequente em algoritmos. Para a notação assintótica, a base do logaritmo não altera a classe de complexidade, desde que seja uma constante maior que 1.

2. Comparação prática do crescimento

Considere n como o tamanho da entrada. A tabela mostra o valor aproximado das funções para diferentes tamanhos, usando lg n= $\log_2$ n.

Complexidade	n = 10	n = 100	n = 1.000
Θ(1)	1	1	1
Θ(lg n)	3,32	6,64	9,97
Θ(n)	10	100	1.000
Θ(n lg n)	33,2	664,4	9.965,8
Θ(n^2)	100	10.000	1.000.000
Θ(n^2 lg n)	332,2	66.438,6	9.965.784,3
Θ(n^3)	1.000	1.000.000	1.000.000.000
Θ(2^n)	1.024	$\approx 1{,}27 \times 10^{30}$	$\approx 1{,}07 \times 10^{301}$

Os valores representam o crescimento matemático das funções, não o tempo real em segundos. Na última linha, a função exponencial cresce tão rapidamente que se torna impraticável para entradas relativamente pequenas.

3. Exemplos de algoritmos para cada classe

Complexidade	***Exemplo simplificado***
Θ(1)	Acessar um elemento de um vetor por índice.
Θ(lg n)	Busca binária em vetor ordenado.
Θ(n)	Percorrer todos os elementos de um vetor.
Θ(n lg n)	Merge Sort.
Θ(n^2)	Comparar todos os pares de elementos, em um algoritmo com dois laços aninhados.
Θ(n^2 lg n)	Executar um trabalho de custo **Θ**(n^2) em cada uma de **Θ**(lg n) etapas.
Θ(n^3)	Três laços aninhados que percorrem n elementos cada.
Θ(2^n)	Enumerar todos os subconjuntos de um conjunto com n elementos.

A complexidade exata depende das operações realizadas e das condições de execução; a quantidade de laços, isoladamente, não é suficiente para determinar a classe assintótica.

Exemplos em Python para as principais classes de complexidade

A seguir, apresenta-se um exemplo de algoritmo para cada uma das oito classes de complexidade assintótica, com código em Python e justificativa da complexidade.

1. Θ(1) — Complexidade constante

Exemplo: acessar um elemento de um vetor por índice.

```
def acessar(vetor, i):
    return vetor[i]
```

Análise: independentemente do tamanho do vetor, a função executa uma quantidade constante de operações para acessar o elemento.

$$T(n) = c \Rightarrow T(n) \in \Theta(1)$$

Exemplo: acessar o elemento de índice 5 em um vetor com 10 ou 1 milhão de elementos.

2. Θ(lg n) — Complexidade logarítmica

Exemplo: busca binária em um vetor ordenado.

```
def busca_binaria(vetor, x):
    inicio = 0
    fim = len(vetor) - 1

    while inicio <= fim:
        meio = (inicio + fim) // 2

        if vetor[meio] == x:
            return meio
        elif vetor[meio] < x:
            inicio = meio + 1
        else:
            fim = meio - 1

    return -1
```

Análise: a cada iteração, o algoritmo descarta aproximadamente metade dos elementos restantes.

$$T(n) \in \Theta(\lg n)$$

Essa complexidade corresponde ao pior caso da busca binária em um vetor ordenado. No melhor caso, quando o elemento está exatamente na primeira posição examinada, o custo é Θ(1).

3. Θ(n) — Complexidade linear

Exemplo: calcular a soma dos elementos de um vetor.

```
def somar(vetor):
    total = 0

    for elemento in vetor:
        total += elemento

    return total
```

Análise: o laço percorre todos os n elementos uma única vez.

$$\mathbf{T(n) = an + b \Rightarrow T(n) \in \Theta(n)}$$

Se o **vetor tiver 100 elementos**, serão realizadas 100 adições ao acumulador.

4. Θ(n lg n) — Complexidade linearítmica

Exemplo: algoritmo Merge Sort, que ordena um vetor dividindo-o em partes menores.

```
def merge_sort(v):
    if len(v) <= 1:
        return v

    meio = len(v) // 2
    esquerda = merge_sort(v[:meio])
    direita = merge_sort(v[meio:])

    resultado = []
    i = j = 0

    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])

    return resultado
```

Análise: o vetor é dividido em duas metades recursivamente, gerando aproximadamente lg n níveis. Em cada nível, o trabalho total de intercalação é proporcional a n.

$$\mathbf{T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) \in \Theta(n \lg n)}$$

A complexidade vale para o **Merge Sort** convencional, considerando o custo de intercalação e cópia das partes.

5. Θ(n²) — Complexidade quadrática

Exemplo: imprimir todos os pares ordenados de elementos de um vetor.

```
def imprimir_pares(vetor):
    n = len(vetor)

    for i in range(n):
        for j in range(n):
            print(vetor[i], vetor[j])
```

Análise: o laço externo executa n vezes e, para cada execução, o interno também executa n vezes.

$$T(n) = n \cdot n = n^2 \Rightarrow T(n) \in \Theta(n^2)$$

Para um vetor de 100 elementos, serão impressos 100 × 100 = 10.000 pares, incluindo os pares em que os dois índices são iguais.

6. Θ(n² lg n) — Complexidade quadrática-logarítmica

Exemplo: executar um algoritmo de custo quadrático em cada etapa de uma sequência que reduz o tamanho do problema pela metade.

```
def exemplo(n):
    tamanho = n

    while tamanho > 1:
        for i in range(n):
            for j in range(n):
                pass

        tamanho //= 2
```

Análise: o laço while executa Θ(lg n) vezes. Em cada iteração, os dois laços internos executam n² vezes.

$$T(n) = n^2 \cdot \lg n \Rightarrow T(n) \in \Theta(n^2 \lg n)$$

Por exemplo, se n = 8, o **while** executa 3 vezes, com 8² = 64 iterações internas em cada etapa, totalizando 192 iterações do bloco interno.

7. Θ(n³) — Complexidade cúbica

Exemplo: percorrer todas as combinações de três índices de um vetor.

```
def percorrer_triplas(vetor):
    n = len(vetor)

    for i in range(n):
        for j in range(n):
            for k in range(n):
                print(i, j, k)
```

Análise: são três laços aninhados, cada um executando n vezes.

$$\mathbf{T(n) = n \cdot n \cdot n = n^3 \;\Rightarrow\; T(n) \in \Theta(n^3)}$$

Para n = 10, o algoritmo imprime 10^3 = 1.000 triplas.

8. Θ(2ⁿ) — Complexidade exponencial

Exemplo: gerar todos os subconjuntos de um conjunto com n elementos.

```
def gerar_subconjuntos(elementos):
    n = len(elementos)

    for mascara in range(2 ** n):
        subconjunto = []

        for i in range(n):
            if mascara & (1 << i):
                subconjunto.append(elementos[i])

        print(subconjunto)
```

Análise: cada elemento pode estar presente ou ausente em um subconjunto. Portanto, existem 2^n subconjuntos. O algoritmo percorre 2^n máscaras, mas cada máscara pode exigir a verificação dos n elementos. Assim, o tempo total desse código é $\Theta(n \cdot 2^n)$, e não $\Theta(2^n)$.

Para obter um exemplo com tempo total $\Theta(2^n)$, podemos contar as máscaras sem percorrer os elementos de cada subconjunto:

```
def contar_mascaras(n):
    for mascara in range(2 ** n):
        pass
```

Esse segundo algoritmo executa $\Theta(2^n)$ iterações.

Resumo comparativo

Notação	Exemplo	Operações dominantes
$\Theta(1)$	Acesso por índice	Constante
$\Theta(\lg n)$	Busca binária	$\lg n$
$\Theta(n)$	Soma de vetor	n
$\Theta(n \lg n)$	Merge Sort	$n \lg n$
$\Theta(n^2)$	Dois laços aninhados	n^2
$\Theta(n^2 \lg n)$	Laços quadráticos repetidos em etapas logarítmicas	$n^2 \lg n$
$\Theta(n^3)$	Três laços aninhados	n^3
$\Theta(2^n)$	Percorrer todas as máscaras	2^n

Observação: a quantidade de laços ajuda a estimar a complexidade, mas é necessário analisar quantas vezes cada operação é executada. A notação Θ representa o crescimento assintótico justo da função de custo, desconsiderando constantes multiplicativas e termos de menor ordem.

Regra fundamental: para n suficientemente grande, uma função de crescimento menor tende a ser mais eficiente quanto ao recurso analisado, desde que os demais fatores sejam comparáveis. **A notação Θ** descreve um limite assintótico justo.

9. Exercício de fixação

1. Um algoritmo acessa vetor[i] diretamente.

a) $\Theta(1)$
b) $\Theta(n)$
c) $\Theta(n^2)$

2. Um algoritmo percorre um vetor de n elementos uma única vez.

a) $\Theta(\lg n)$
b) $\Theta(n)$
c) $\Theta(n^2)$

3. Um algoritmo divide repetidamente o problema pela metade até chegar a um único elemento.

a) $\Theta(1)$
b) $\Theta(\lg n)$
c) $\Theta(n)$

4. Dois laços aninhados percorrem n elementos cada.

a) $\Theta(n)$
b) $\Theta(n^2)$
c) $\Theta(n^3)$

5. Um algoritmo enumera todas as combinações possíveis de escolhas sim/não para n elementos.

a) $\Theta(n^2)$
b) $\Theta(n \lg n)$
c) $\Theta(2^n)$